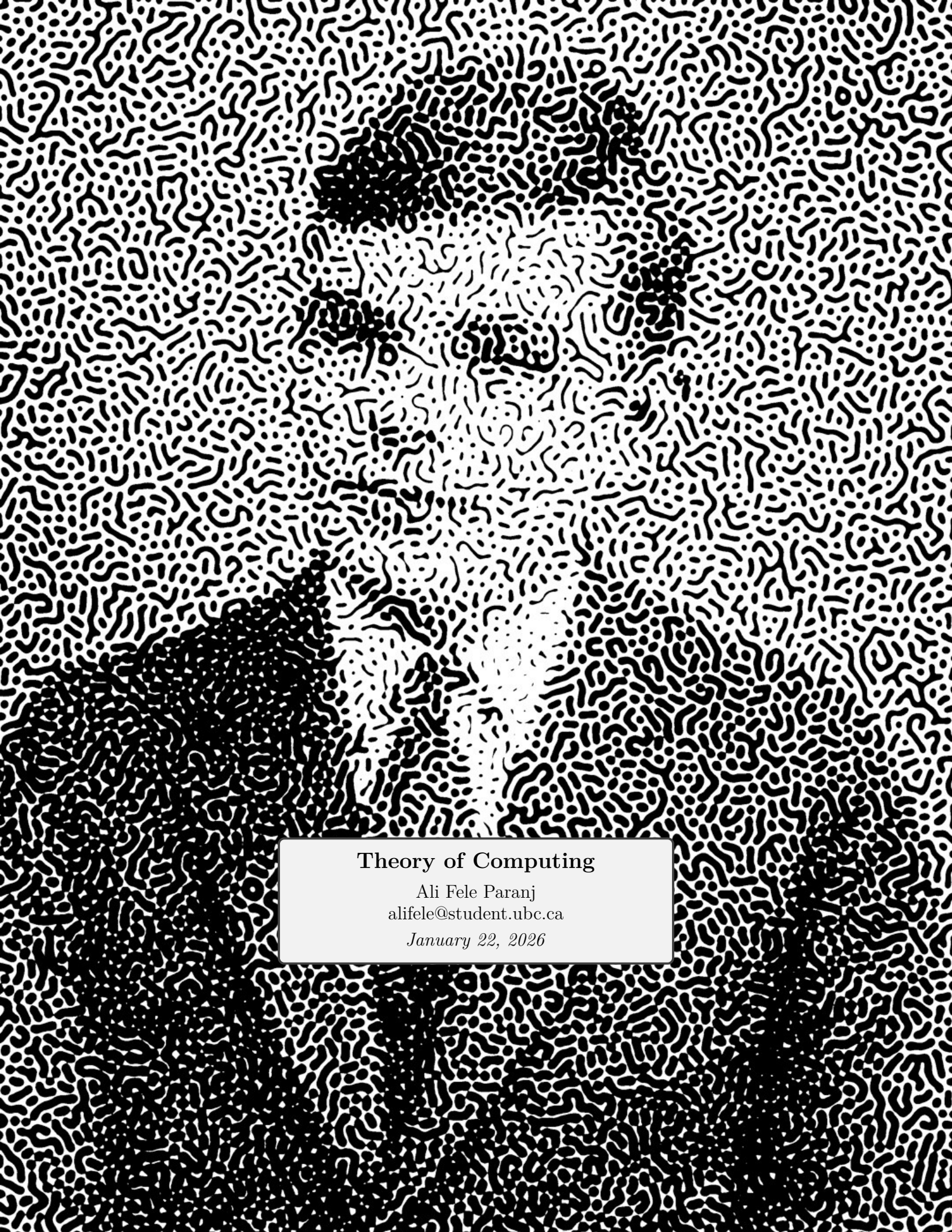


Theory of Computing

Ali Fele Paranj
alifele@student.ubc.ca

January 22, 2026

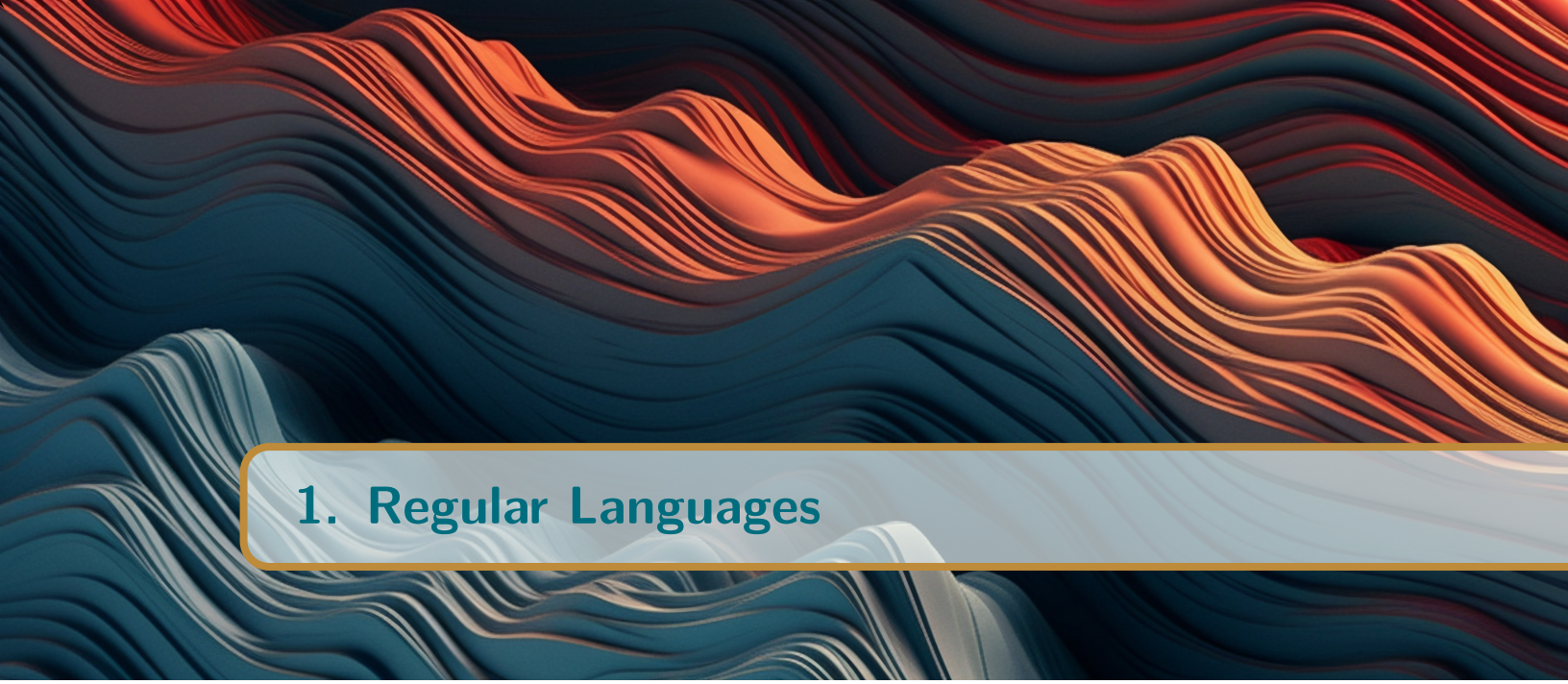
0. Contents

1 Regular Languages **5**

1.1 DFA Over Single Alphabet Languages 6

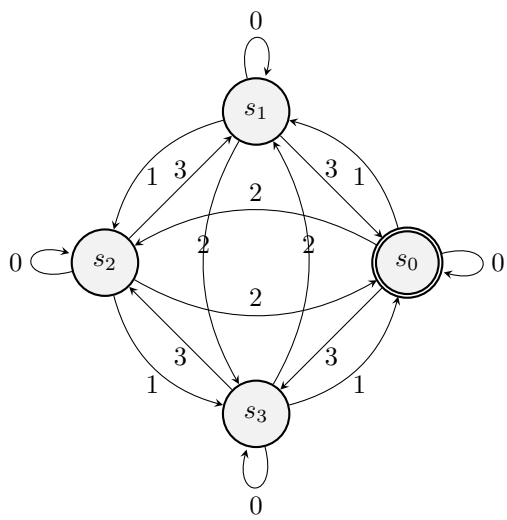
2 Decidability **7**

2.1 Problems 7



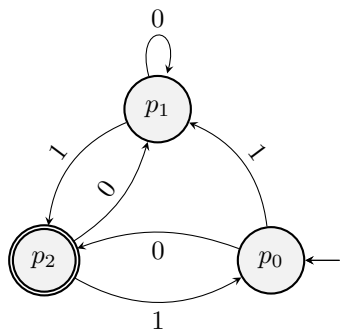
1. Regular Languages

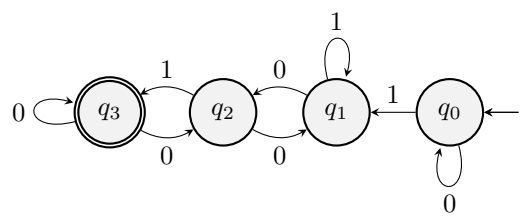
Observation 1.1 The transition rules of a finite automaton can formally shown as $\Sigma \times S \to S$. When Σ is a group, then some transition rules are actually group actions (satisfies the conditions for being a group action). When $\Sigma = S = G$ for some group, then interesting stuff can happen. For instance, let $\Sigma = S = \mathbb{Z}_4$ cyclic group of order 4. Then the resulting finite automta can accept the strings that their sum is mod 4. Then the transition rules will be as below. This finite state machine can accept the strings that their numerical sum of the values of the string is 0 modulo 4.



Cartesian product of finite automaton

■ **Example 1.1** Let M_1 and M_2 be finite automota, and let L_1 and L_2 be the languages that they recognize respectively. Then $M_1 \oplus M_2$ is the finite automoton that recognizes $L_1 \cup L_2$. For instance, consider the following FSM's:





The state space of $M_1 \oplus M_2$ is $S_1 \times S_2$ and the transition functions is $\delta = \delta_1 \otimes \delta_2$, given by

$$\delta((p,q),s) = \delta_1 \otimes \delta_2((p,q),s) = (\delta_1(p,s), \delta_2(q,s)).$$

■

1.1 DFA Over Single Alphabet Languages

Let $\Sigma = \{a\}$ be a set consisting of one symbol. Any DFA $(Q, \Sigma, \delta, q_0, F)$ will have

2. Decidability

Theorem 2.1 — Halting problem is undecidable. The language

$$A_{TM} = \{\langle T, \omega\rangle \mid T \text{ accepts } \omega\},$$

is undecidable.

Informal proof. There are multiple ways to show this, but some of these methods often gets cluttered with many intermediate definitions and sometimes mind bending self references. Here I will take a simple approach. Assume to the contrary that A_{TM} is decidable. I.e. $\exists \eta \in TM$ such that η decides A_{TM}. Informally speaking, the problem with the existence of such Turing machine is that the negated variants of this machine, denoted by $\bar{\eta}$, has a special power that is forbidden by the Cantor's diagonalization argument. I.e. it can accept the Turing machines that do not accept themselves. To put in other words, define $D : \Sigma^* \to 2^{\Sigma}$, where for $\omega \in \Sigma^*$, $D(\omega)$ is the language that ω recognizes ($D(\omega) =$ if ω is not a Turing machine). Let $\alpha \in TM$ such that $\alpha \notin D(\alpha)$, i.e. α does not accept itself. So $\langle \alpha, \alpha\rangle \notin A_{TM}$. So $\bar{\eta}$ accepts $\langle \alpha, \alpha\rangle$. This implies that $T = \{\omega \in \Sigma^* \mid \omega \notin D(\omega)\} \subset D(\bar{\eta})$. This is a contradiction by the Cantor's theorem. □

2.1 Problems

■ **Problem 2.1 — Sipser.** Consider the problem of determining whether a DFA and a regular expression are equivalent. Express this problem as a language and show that this is decidable.

Solution Consider the following language

$$L = \{\langle D, R\rangle \mid D \in \mathcal{D} \text{ is equivalent to } R \in \mathcal{R}\},$$

where we have used th notation $\mathcal{D}$ and $\mathcal{R}$ to denote the set of all DFA and regular expressions respectively. Let $\langle D, R\rangle \in L$ and denote their corresponding language by $\mathcal{L}(D)$ and $\mathcal{L}(R)$. D is equivalent to R if and only if $\mathcal{L}(D) = \mathcal{L}(R)$ if and only if $\mathcal{L}(D)\Delta\mathcal{L}(R) = \varnothing$, where Δ denotes the symmetric difference. Since both of the languages above are regular languages, and the set of regular languages is closed under symmetric difference (follows from the set of regular languages being closed under the elementary operations defining symmetric difference), $L_\Delta = \mathcal{L}(D)\Delta\mathcal{L}(R)$ is a regular language. So $\exists C \in \mathcal{D}$ such that $\mathcal{L}(C) = L_\Delta$. Using the fact that $E_{DFA} = \{D \in$

$\mathcal{D} \mid L(D) = \varnothing\}$ (i.e. the set of DFAs with empty language) is decidable, we can decide if $\mathcal{L}(C) = \varnothing$, hence we can decide if $\langle D, R\rangle \in L$, hence L is decidable.

■ **Problem 2.2 — Sipser.** Let $ALL_{DFA} = \{\langle A\rangle | A \text{ is a DFA and } \mathcal{L}(A) = \Sigma^*\}$. Show that ALL_{DFA} is decidable.

Solution First, consider the following identity

$$(\mathcal{L}(A))^c = \mathcal{L}(A^c),$$

where A^c is a new DFA that its accept states are complement to that of A. So we can write

$$ALL_{DFA} = \{\langle A\rangle | \mathcal{L}(A) = \Sigma^*\} = \{\langle A\rangle | \mathcal{L}(A)^c = (\Sigma^*)^c\} = \{\langle A\rangle | \mathcal{L}(A^c) = \varnothing\}.$$

So given $A \in ALL_{DFA}$:

(i) convert A to A^c,

(ii) decide if $\mathcal{L}(A^c) = \varnothing$,

(iii) decide if $\mathcal{L}(A) = \Sigma^*$.

■ **Problem 2.3 — Sipser.** Let $E_{TM} = \{\langle M\rangle | M \text{ is a TM and } \mathcal{L}(M) = \varnothing\}$. Show that E^c_{TM} is Turing-recognizable.

Solution Let $m \in E^c_{TM}$. Then $\mathcal{L}(m) \neq \varnothing$. So $\exists w \in \Sigma^*$ such that m accepts w. Put the elements of Σ^* in lexicographical order $s_1, s_2, \cdots,$. w should appear somewhere in this sequence, say s_p for some $p \in \mathbb{N}$. Then do the following

(i) Run i steps of m on $s_1, \cdots, s_i$ for each $i \in \mathbb{N}$.

Then m will eventually halt as m accepts s_p in N_p steps.